THE CULPRIT

CI/CD & Secrets

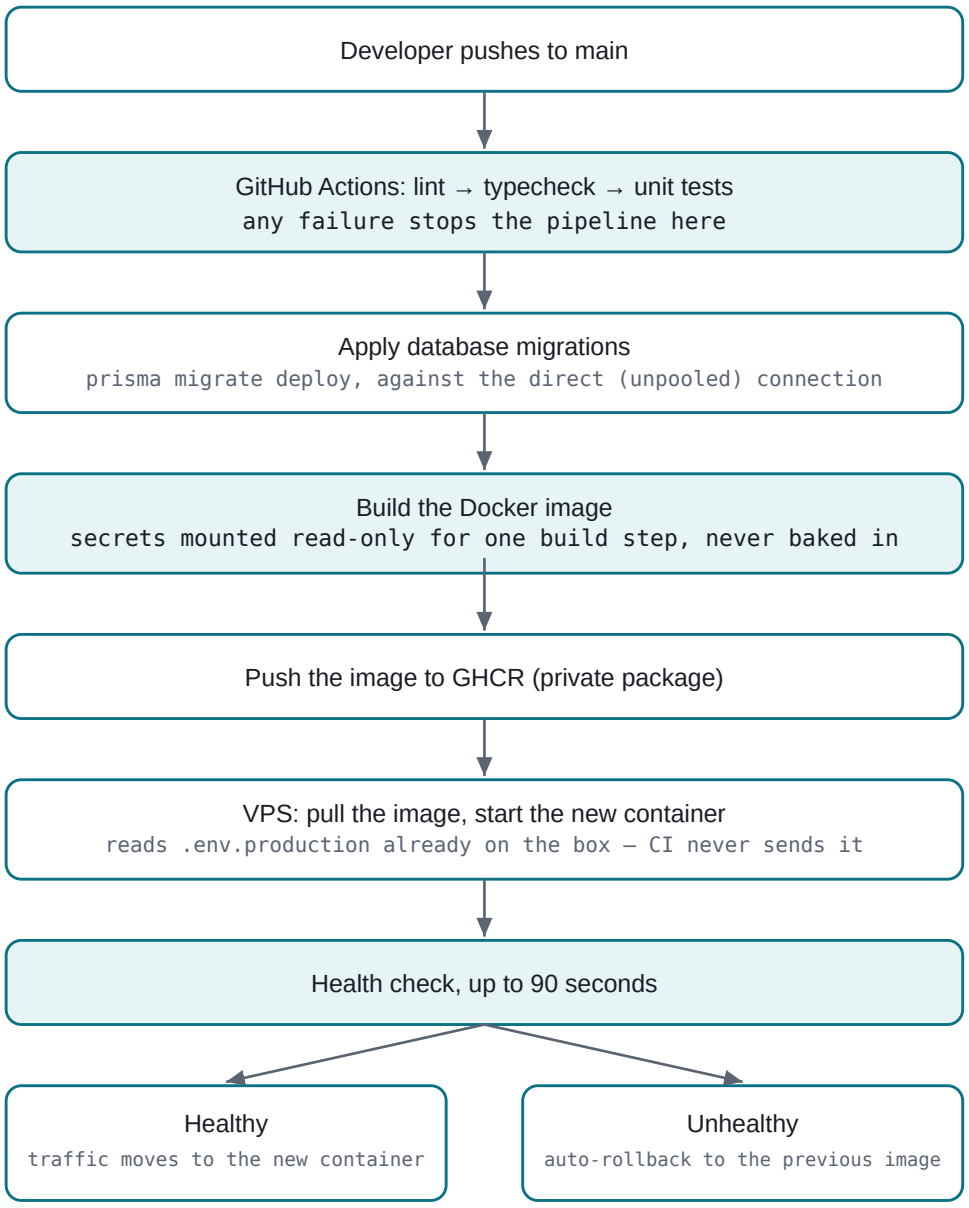
How a code change goes from a laptop to the live site automatically, and how the team shares database passwords, API keys and other credentials along the way without ever putting them in a chat message, an email, or the repository itself.

`Pipeline: GitHub Actions` `Local secrets: Doppler` `Repo visibility: Public`

`Reviewed: 13 Aug 2026`

01 How a Deploy Happens

Nobody deploys by hand. Pushing to `main` is the only trigger — everything after that is automatic, and every step has to succeed before the next one runs. A test failure or a bad migration stops the pipeline before a single line of the change reaches the live site.



Every arrow is a hard gate — nothing downstream runs unless the step above it succeeded. A stuck migration or a failed health check both leave the previous, working version exactly as it was.

The VPS itself only ever runs two commands, `docker pull` and `docker compose up`. It never installs a package, never runs `npm`, never compiles anything — all of that already happened in CI, on a machine built for it, not on the small box actually serving traffic.

02 Where Config Values Live

There is no single place secrets are "shared." Four different tools each hold exactly the values the step they cover needs, and nothing more — a developer's laptop never sees the production database password just to run the app locally, for instance, unless they specifically need to.

STAGE	TOOL	WHAT IT HOLDS	WHO CAN READ THE RAW VALUE
Local development	Doppler	Every var, per developer, injected at process start	Whoever Doppler's project access grants — not stored on disk as a file
CI (build & test)	GitHub Actions Secrets & Variables	Same production values, scoped to this one repository	Any workflow run in this repo; masked in logs automatically
Docker image build	BuildKit secret mounts	Just the three values `next build` needs to even trace the code	Nobody after the build — mounted read-only for one layer, never written to the image
Production runtime	`.env.production` file on the VPS	Every runtime value the running container actually uses	Whoever can log into that VPS — CI never writes to this file

The important design choice is the last row: CI builds and deploys the app, but it never touches `.env.production`. That file was written once, by hand, when the server was first set up, and only a human editing it directly changes it. A compromised or misconfigured CI run literally cannot overwrite or exfiltrate it, because CI never has a copy of it in the first place.

03 Every Value, Categorized

None of the values below are shown — only their names and what kind of thing they are. Two kinds exist in GitHub Actions, and mixing them up matters: a **Secret** is encrypted at rest and hidden from every log; a **Variable** is plain repo configuration, visible to anyone with read access to the repo, used for things that were never sensitive to begin with (a public site URL, a hostname).

NAME	KIND	WHY IT'S THAT KIND
`DATABASE_URL`	Secret	Pooled Postgres connection string — includes a password
`DIRECT_URL`	Secret	Unpooled Postgres connection string — same reason
`BETTER_AUTH_SECRET`	Secret	Signs every session cookie; whoever holds it can forge admin sessions
`TURNSTILE_SECRET_KEY`	Secret	Server-side key that verifies bot-check tokens
`DEPLOY_SSH_KEY`	Secret	Private key for the VPS's dedicated `deploy` user
`RESEND_API_KEY`	Secret	Sends email as this project — optional, unused today
`R2_ACCESS_KEY_ID` / `R2_SECRET_ACCESS_KEY`	Secret	Write access to the file storage bucket
`CLOUDFLARE_API_TOKEN`	Secret	Scoped narrowly (Cache Purge only, one zone), still a live credential
`NEXT_PUBLIC_APP_URL`	Variable	Ends up in the client bundle either way — it's the site's own public URL
`NEXT_PUBLIC_CALENDLY_URL`	Variable	The public booking page link, shown to every visitor
`NEXT_PUBLIC_TURNSTILE_SITE_KEY`	Variable	Turnstile's site keys are meant to be public; only the secret key isn't
`R2_PUBLIC_URL`	Variable	The bucket's public read URL — the whole point is that it's public
`DEPLOY_HOST` / `DEPLOY_USER`	Variable	An SSH destination, not a credential — the key is what actually grants access
`ADMIN_EMAIL` / `ADMIN_INITIAL_PASSWORD`	Runtime only	Never touches CI at all — set once directly in `.env.production`, rotated after first login

Worth noticing: every genuinely sensitive value is a `Secret`, and nothing sensitive was ever filed as a `Variable` for convenience. That split is the one place a small mistake would matter most, and it's drawn correctly on every value checked above.

04 Is This Secure to Share?

Checked against the actual repository — its full commit history, its workflow files, and its access list — not assumed from how the pipeline is supposed to work.

What's done right

PRACTICE	WHY IT MATTERS
No secret has ever been committed	Checked the full git history, not just the current files — every commit, every branch. Only the placeholder `.env.example` was ever added; every real value stayed out from the very first commit. This matters more than usual here because the repository is **public** — a secret in history would already be exposed to anyone.
Pull requests can't read secrets	The pipeline only triggers on a direct push to `main` or a manual run — never on `pull_request`. That closes off the most common way a public repo leaks CI secrets: an outside contributor opening a PR from a fork that tries to print them.
Build secrets never enter the image	The three database/auth values `next build` needs are mounted read-only for a single build step (Docker BuildKit secrets), not passed as a plain build argument. A plain argument would sit permanently in the image's history, recoverable with `docker history` by anyone who can pull it; a mounted secret leaves no trace.
Secrets and public config are kept separate	See the table above — nothing sensitive was filed as a plain repo Variable to save a step.
Local dev never uses a shared `.env` file	Doppler injects values into the process at start; nothing lands on disk as a plain-text file to begin with, so there's no file to accidentally attach to a message, commit, or leave on a laptop.
Deploy access is narrowly scoped	The SSH key CI uses belongs to one dedicated `deploy` user on the VPS, not a personal account. The registry login on the server uses that run's own short-lived `GITHUB_TOKEN`, which expires shortly after the job ends — nobody provisioned a permanent registry password on the box.
A bad deploy can't stay live	Not a secrecy property, but a safety one: the health check + automatic rollback means a broken release (including one caused by a misconfigured secret) reverts itself instead of staying up while someone investigates.

Residual risks — worth knowing, not necessarily worth panicking about

RISK	DETAIL
Registry token passed on an SSH command line	The VPS logs into GHCR via `echo "$TOKEN" | docker login --password-stdin`, run as a remote SSH command. For the moment that command executes, the token is visible in that process's argument list to anything else on the shared box. Impact is limited — the token is scoped to one workflow run and expires within minutes — but it's a real, specific exposure window worth naming rather than assuming away.
`.env.production` is a plain-text file	No encryption at rest, no access log, no rotation reminder — its safety depends entirely on VPS file permissions and who can SSH in. The deployment docs don't currently state it should be `chmod 600`, owner-only.
Anyone with repo write access can effectively read any CI secret	This is how GitHub Actions works everywhere, not a flaw specific to this project — a collaborator with write access could edit a workflow to print or exfiltrate a secret's value, and GitHub's log-masking only catches the exact string, not a re-encoded copy of it. Worth naming because it means the real access boundary for "who can see production secrets" is *repo write access*, not "the CI system." Three accounts currently hold write or admin access to this repository.
Branch protection on `main` wasn't verified	Not confirmed from what's visible in the repository itself whether direct pushes to `main` (bypassing review) are currently possible for a collaborator. Doesn't change the secrets picture directly, but it's the backstop that would normally catch a malicious or accidental workflow edit before it reaches `main` and runs with real credentials.
The shared VPS is a shared blast radius	This app's container is one of several on the same box. Isolation between them is Docker's container boundary plus file permissions, not separate virtual machines — a severe compromise of a neighboring app's container, or of the host itself, would put `.env.production` at risk regardless of anything this project does right.

> Short answer: yes, reasonably. Nothing sensitive has ever reached the public repository, the pipeline is built so a compromised pull request can't read secrets, and build-time credentials leave no recoverable trace in the image. The residual risks above are the normal cost of running a small, low-budget pipeline rather than signs of something done carelessly — each one has a cheap, specific fix, listed at the end of this page.

05 Joining the Team

What a new contributor actually needs depends on what they're doing — nobody needs everything below just to open the project.

TO DO THIS…	…YOU NEED
Run the app locally, make code changes	A GitHub collaborator invite, plus Doppler project access (`culprit` → `dev` config). Nothing else — `npm run dev` pulls every value it needs automatically.
Merge to `main`, trigger a real deploy	Write access to the repository. No separate deploy credential — pushing to `main` is the entire deploy trigger.
Debug the live server directly	An SSH account on the VPS itself, provisioned separately from GitHub access. Most debugging (logs, health, recent deploys) doesn't need this — see Deployment.
Rotate a production secret	Doppler access (to update the value everyone else pulls) *and* either GitHub Secrets access or VPS SSH access, depending on whether the value is used at build time or only at runtime.

Nobody needs the VPS SSH key to deploy — that key lives only inside GitHub Actions and is used by the pipeline itself, never handed to a person.

06 Open Recommendations

None of these are urgent — the pipeline is sound today — but each closes one of the residual risks above for close to zero ongoing cost.

- **Confirm `main` is a protected branch** requiring review before merge, so a workflow-file change (the one path that can reach a real secret) always gets a second set of eyes.
- **Turn on GitHub push protection for secret scanning** (Settings → Code security). Scanning itself is already on by default for public repositories at no cost; push protection additionally blocks a matching commit before it's ever pushed, rather than only flagging it afterward.
- **Set `.env.production` to `chmod 600`**, owned by the deploy user only, and say so explicitly in `docs/deployment/docker-vps.md` so it isn't left to whoever set the box up to remember.
- **Periodically review Doppler project membership** the same way repo collaborators get reviewed — it's the one access list in this whole pipeline that doesn't live in GitHub, so it's the easiest one to forget.